CS 684 Embedded System

Lab 1: Statechart for the Line Following Robot

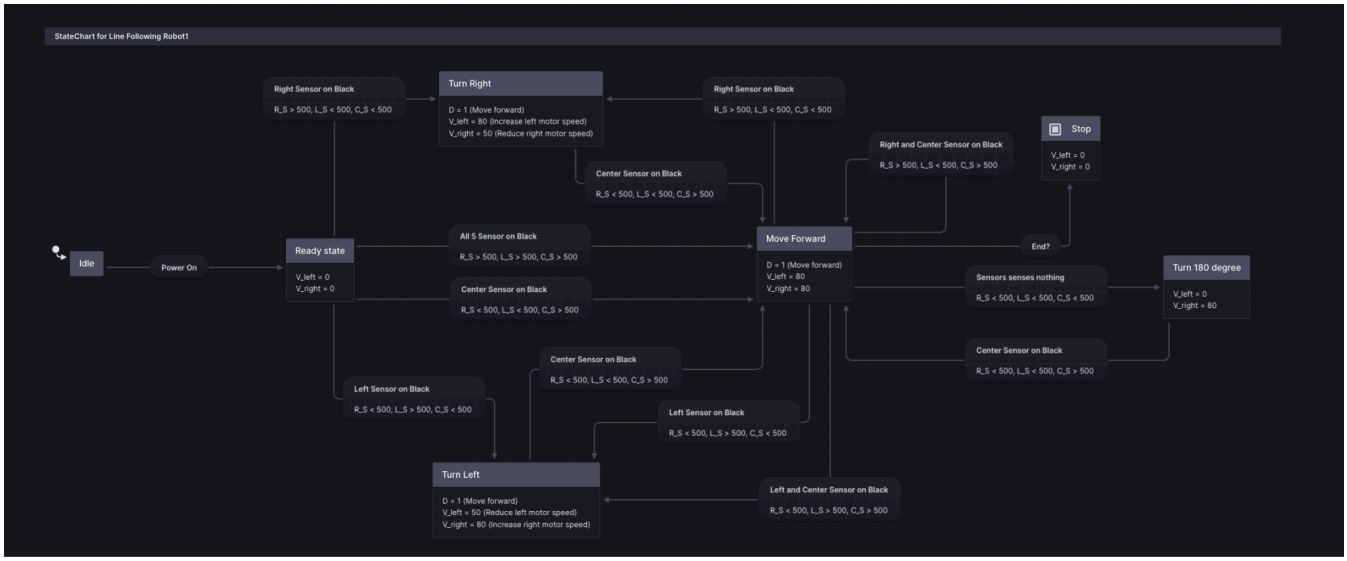
Group: CodeOnBoard

Utkarsh (24M2122)

Ritika (24M0855)

Jayesh (23M0805)

Atul (23M0764)



1. **Statechart Link:** [statechart_link]
2. **YouTube Link:** [youtube_link]
3. **Strategy:**
   - Five sensors are integrated into the robot to detect the path and take appropriate actions, the end goal is to reach the end mark.
   - Since sensor values range from 0-1023, we have considered any readings below 500 to represent white surface and readings above and including 500 as black surface.

- We have assigned priorities to different sensors in order to make a decision in case multiple sensors show reading.
- Our statechart has the following states:
    - **Idle State:** The robot is powered off in this state.
    - **Ready State:** This is the initial state of the robot after it is powered on.
    - **Stop State:** When we reach the end line, we go to the stop state.
    - **Move Forward:** In this state, our robot moves forward, until the sensor readings change.
    - **Turn Left:** In this state, our robot takes a left turn, whenever our readings change and we decide to take the path on the robot's left.
    - **Turn Right:** In this state, our robot takes a left right, whenever our readings change and we decide to take the path on the robot's right.

4. Assumption & Description:

- We have assumed that the position of the sensors is such that:
    - The two sensors to the extreme left will show values greater than or equal to 500 when there is a black surface to the left.
    - The two sensors to the extreme right will show values greater than or equal to 500 when there is a black surface to the right.
    - The central sensor will show a value greater than or equal to 500 when there is black surface ahead.

- We have also assigned priority to the different sensors, the left sensor has greater priority than the central sensor and the central sensor has greater priority than the right sensor. This priority helps us decide when there are multiple options to choose from, to finally reach the end goal. For instance, let’s say at a juncture both the two left sensors and the central sensor shows reading greater than or equal to 500, then our robot will take a left turn since the priority of the left sensors is the highest.

5. Transitions in the statechart:

The following table shows the transitions in the statechart based on sensor readings

0 represents sensor values lesser than 500

1 represents sensor values greater than or equal to 500

Left Sensor	Center Sensor	Right Sensor	Description
0	0	0	No line detected
0	0	1	Right sensor shows reading, robot turns right
0	1	0	Central sensor shows reading, move forward
0	1	1	Central & right sensors show reading, move forward
1	0	0	Left sensor shows reading, robot turns left
1	0	1	Left and right sensors show reading, robot turns left
1	1	0	Left and Central sensors show readings, robot turns left
1	1	1	Left, Central and Right sensors show readings, robot turns left.

6. Contribution

Every team member contributed equally while making the statechart. We regularly engaged in meetings to plan and improvise the flow of our robot and finally came up with a final statechart diagram for our robot. Two of our team members were given the task of recording and editing the youtube video, while the other two team members worked on the document.

Lab 2: Line Following Robot using PID and State-charts

Group: CodeOnBoard
Members:
Utkarsh (24M2122), Ritika (24M0855), Jayesh (23M0805), Atul (23M0764)

1. YouTube Link

`https://youtu.be/iVlN_QFH49U`

2. Assumptions

1. The robot follows a white line on a black surface; the threshold to detect white line detection is set to 500. If the sensor value is less than the white line threshold, then it is considered as active.
2. The robot can be in the following states: Start, Forward, Stop, DriftLeft, DriftRight, Left, Right, and Turn180deg.
3. To control the direction of movement we have assumed PID correction values between the range (-100 to 100).
4. We have defined ranges of direction based on it, the decision of Moveforward/DriftLeft/Left/DriftR state is made.
    - If pidValue is between [-30 to 30] then it goes to Moveforward
    - If pidValue is between [30 to 60] then it goes to DriftLeft
    - If pidValue is between [-60 to -30] then it goes to DriftRight
    - If pidValue is greater than 60 then it goes to Left
    - If pidValue is lesser than -60 then it goes to Right
    - If all sensors active then it goes to Stop
    - If all sensors are inactive then it goes to Backward
5. In the Backward state, the robot rotates 180 degrees and then transitions only to the Forward state.
6. Once the robot reaches the Stop state, it remains in this state.

3. Description

1. **MoveForward:** When the pidValue is in [-30 to 30] range, indicating the robot is aligned with the white line.
2. **Drifting Left:** When the pidValue is in [30 to 60] range, indicating the robot is requiring a small leftward correction.
3. **Drifting Right:** When the pidValue is in [-60 to -30] range, indicating the robot is requiring a small rightward correction.

4. **Left Turn:** When the pidValue is greater than 60, indicating the robot needs to turn left immediately.
5. **Right Turn:** When the pidValue is lesser than -60, indicating the robot needs to turn right immediately.
6. **Stop Condition:** When all sensors detect white, meaning the robot should stop.
7. **Backward Turn:** When all sensors detect black, meaning the robot is off-track and must turn around 180 degrees.

4. Contribution

- **Jayesh:** Worked on PID Controller, Start, Stop, Turn180 States and Video.
- **Utkarsh:** Worked on PID Controller, Start, Stop, Turn180 States.
- **Ritika:** Worked on Drift Left, Drift Right, Left and Right States, Start, Stop, Turn180 States.
- **Atul:** Did Simulation, Testing, Readme, and Start, Stop, Turn180 States.

Lab 3: Line Following Robot Refinement

Group: CodeOnBoard
Members:
Utkarsh (24M2122), Ritika (24M0855), Jayesh (23M0805), Atul (23M0764)

1. YouTube Link

`https://youtu.be/o8YfFOumMso`

2. Strategy and Refinement

In this lab, the primary focus was to refine the robot's logic for better stability and accurate line following:

- Planned and refined the calculation of PID values.
- Adjusted motor velocities in different states (Forward, Drift, Turn) to ensure the robot moves smoothly.
- Successfully resolved issues such as incorrect robot movement and unexpected stops while following the line through consistent testing.

3. Contribution

Every team member contributed equally while building the line-following robot. We regularly held meetings to plan and refine the calculation of PID values, adjust motor velocities in different states, and ultimately finalize the implementation. We encountered issues such as incorrect robot movement and unexpected stops while following the line. However, through consistent testing and discussions in our meetings, we successfully resolved these challenges.

Lab 4: Blackline Following and Obstacle Avoidance

Group: CodeOnBoard
Members:
Utkarsh (24M2122), Ritika (24M0855), Jayesh (23M0805), Atul (23M0764)

1. YouTube Link

`https://youtu.be/n76jOY1SbKU`

2. Description of States

- **Blackline Following:**
  - **BlackForward:** The robot moves to this state after the switch_condition is true. This state means the bot is moving in the forward direction on the black line.
  - **BlackStop, BlackRight, BlackLeft, BlackDriftRight, BlackDriftLeft:** These states are triggered to stop on the black line, move right, move left, drift right, and drift left on the black line respectively, according to sensor conditions.
- **Obstacle Avoidance:**
  - **ObstacleAvoidance:** Triggered when the front IR sensors detect an obstacle. The robot pauses, takes a right turn (RightTurn1), and moves forward (Move1) until the left IR sensor senses the obstacle.
  - It then moves a bit forward (Move2), takes a left turn (LeftTurn1), and moves forward again until the left IR sensor senses the object on its left.
  - It moves a bit more to have enough space to turn, takes another left turn (LeftTurn2) to face the black line again, moves forward until the black line is detected, and finally turns right to resume moving on the black line.

3. Contribution

- **Ritika and Jayesh:** Worked on Blackline switching logic and blackline following, along with the counter node. They worked on node detection, BlackForward, BlackStop, BlackLeft, BlackRight, BlackDriftLeft, BlackDriftRight, and worked out the logic for the bot to turn right on both the first two nodes detected, turn left on the third node, and then stop on the fourth node detected.
- **Atul and Utkarsh:** Worked together on the obstacle detection and avoidance logic. They wrote ObstacleAvoidance, RightTurn1, Move1, Forward1, Move2, LeftTurn1, Move3, Forward2, Move4, LeftTurn2, Move5 and RightTurn3 states, designing the algorithm to avoid the obstacle and safely return to the black line path.

Lab 5: Final Line Following Robot with Parking and Obstacle Avoidance

Group: CodeOnBoard
Members:
Utkarsh (24M2122), Ritika (24M0855), Jayesh (23M0805), Atul (23M0764)

1. YouTube Link

`https://youtu.be/dlal6s02jiA`

2. Strategy and Refinement

The final phase involved integrating all previous functionalities into a complete, robust system:

- Implemented and refined the calculation for the switchcondition.
- Perfected Obstacle Avoidance System (OAS) logic.
- Integrated Parking logic to ensure the robot stops correctly at the end of the course.
- Adjusted motor velocities across all states to optimize performance.
- Addressed and resolved critical issues including incorrect robot movement, obstacle collision, incorrect parking, and unexpected stops while following the line.

3. Contribution

Every team member contributed equally while building the line-following robot. We regularly held meetings to plan and refine the calculation of switchcondition, OAS logic, and Parking logic, adjust motor velocities in different states, and ultimately finalize the implementation.

- We encountered issues such as incorrect robot movement, obstacle collision, incorrect parking and unexpected stops while following the line. However, through consistent testing and discussions in our meetings, we successfully resolved these challenges.
- **Jayesh and Atul:** Created and recorded the simulation and code explanation videos.
- **Ritika and Utkarsh:** Created the presentation and edited the video.